

Expression Conversion

Infix: $A+B$
 operator is in between operands.

Postfix: $AB+$
 operator is placed after operands

Prefix: $+AB$
 operator is placed before operands.

① Convert infix to Postfix

$A+B$:

input	expression	stack
A ✓	A ✓	#
+ ✓	A	+
B	AB ✓	+
	AB+ ✓	

$A+B-C$:

input	expression	stack
A ✓	A ✓	#
+ ✓	A	+ ✓
B	AB ✓	(+)
(-)	AB+ ←	-
C ✓	AB+C	(-)
	AB+C(-) ✓	

same precedence

Precedence:

1. () parenthesis ✓
2. ^ power operator. ✓
3. *, / , % Multiply, Division, Modulus ✓
4. +, - Plus, Minus. ✓

$A+B \times C$:

input	expression	stack
A	A ✓	#
+	A	+
B ✓	AB ✓	(+)
(*) ✓	AB	+, *
C	ABC ✓	+, *, C
.	<u>ABC*+</u>	#

$A * (B + C)$:

input	expression	stack
A	A ✓	#
*	A	* ✓
(	A	*, (✓
B	AB ✓	*, (, B
+	AB	*, (, + ✓
C	ABC+	*, (, +, C
C ✓	ABC ✓	*, (, +, C
) ✓	ABC+ ✓	*
.	<u>ABC+*</u>	

$5 * A^{\wedge}(C * B)$:

input	expression	stack
5	5 ✓	#
*	5	* ✓
A ✓	5A ✓	*
^ ✓	5A	* ^ ✓
(✓	5A	* ^ (✓
C	5AC ✓	* ^ (, C
*	5AC	* ^ (, *, C
B ✓	5ACB ✓	* ^ (, *, C, B
) ✓	5ACB* ✓	* ^ (, *, C, B,)
↓	<u>5ACB*^*</u>	

f) Convex-t

to prefix.

$A+B:$	input	stack	Expression	Stack
	B ✓	B		#
	+ ✓	B		+ ✓
	A ✓	BA ✓		(+)

BA+ ✓

Expression: $+AB$

$A+B-C:$	input	stack	Expression	Stack
	C ✓	C		#
	- ✓	C		- ✓
	B ✓	CB ✓		(-)
	(+)	CB- ✓		+
	A ✓	CB-A ✓		(+)
		CB-A+ ✓		

Expression: $+A-BC$ Prefix

$A+B*C:$	input	stack	Expression	Stack
	C ✓	C		#
	* ✓	C		* ✓
	B ✓	CB ✓		(*)
	(+)	CB* ✓		(+)
	(A)	CB*A ✓		(+)
		CB*A+ ✓		

Expression: $+A*BC$ Prefix

input	Stack Expression	Stack
)		) ✓
c ✓	c ✓	)
+ ✓	c	) + ✓
B ✓	CB ✓	) +
(✓	CB + ←	
* ✓	CB +	* ✓
A ✓	CB + A ✓	(*)
	CB + A * ←	

Expression: *A + BC ✓

$5 * A^{\wedge} (C * B)$:

input	Stack Expression	Stack
)		)
B	B ✓	)
*	B	) * ✓
C ✓	BC ✓	(*)
(✓	BC * ←	
^	BC *	^
A ✓	BC * A ✓	(^)
(*)	BC * A ^ ←	*
5 ✓	BC * A ^ 5 ✓	(*)
	BC * A ^ 5 * ←	

Expression: * 5 ^ A * C B

prefix

Expression Evaluation

infix: $4 * (5 - (7 + 2)) = 4 * (5 - (9)) = 4 * (5 - 9) = 4 * (-4) = -16$

Postfix: $4 5 7 2 + - *$

input	op1	op2	result	stack	operator
4 ✓				4 ✓	
5 ✓				4, 5 ✓	
7 ✓				4, 5, 7 ✓	
2 ✓				4, 5, 7, 2 ✓	
+	7	2	9	4, 5, 9	Pop → op2
-	5	9	-4	4, -4	φ
*	4	-4	-16	-16	
→	Ans: -16 ✓				

op1 () op2
↓
operator

A + B
↑ ↑
op1 op2

Pop → op2
φ

infix: $((3 + 4) * 2) / 7$

Postfix: $3 4 + 2 * 7 /$

input	op1	op2	result	stack	operator
3 ✓				3, ✓	
4 ✓				3, 4 ✓	
+	3	4	7	7	(op1) operator (op2)
2 ✓				7, 2 ✓	
*	7	2	14	14	
7 ✓				14, 7 ✓	
/	14	7	2	2	
→	Ans = 2 ✓				

Prefix Evaluation

Prefix: $-1*2*5+3652$

$(2563+8*2*1-)$

input op1 op2 result stack.

2 ✓

2 ✓

5 ✓

2, 5 ✓

6 ✓

2, 5, 6 ✓

3 ✓

2, 5, 6, 3 ✓

+ ✓

3

6

9 ✓

2, 5, 9

5 ✓

2, 5, 9, 5 ✓

* ✓

5

9

45 ✓

2, 5, 45

2 ✓

2, 5, 45, 2 ✓

* ✓

2

45

90 ✓

2, 5, 90

1

90

5

18 ✓

2, 18

Top → op¹

-

18

2

16

16

2

18

-16

Ans : 16

✓